Основни типове: Int, Integer, Double, Bool, Char, String | **String** == [Char]
Аритметика: +, -, *, / (дробно деление –не гърми с Integer), div, mod
Логика: &&, ||, not, ==, /=, <…
Условия : = if …. then …. else или гардове: | … = … | otherwise = …
let … in… или **where**…
Тип функции: name :: Type -> ReturnType
lists -> (x:xs) или (x:y:xs) | **helper** n acc | **[1..**10**]** [1,3..10] | [x | x <- …, …] | **Рекурсия:** ДЪНО
(a,b) -> fst = a, snd = b (не е нужно да са от 1 тип)
Ламбда: \x -> …. (връща функция) | g . f (**композиция** – f(g(x))) | **currying** (+2), uncurry
map *func list* , **filter** *predicate list*, **foldr/l** *func* (a->b->b) *acc list* | **import** Data.List (nub) | fromEnum, toEnum, fromIntegral
Ord *a* => (типът данни може да бъде подреден) | **Eq** *a* => (сравним тип)
Prelude:

- Числови: succ (следващ) , pred (предишен), abs, signum, min, max, minimum, maximum, gcd, subtract
- Проверки: even, odd, elem *el list*, null (? празен), any(дали някой ел. от списъка отговаря на условие), all (дали всички ел. от списъка отговарят на условие), compare *x y == GT/LT/EQ*
- Агрегиращи: length, sum, product, minimum, maximum, reverse
- Достъп: head, tail, last, init (всички без последния), *list* !! *i* (ел. на индекс i)
- Подсписъци: take *n list* (първите n), drop *n list* (без първите n), takeWhile, dropWhile
- Комбиниране: (:), (++), concat, zip *l1 l2* (($l1_1$, $l2_1$)...), unzip
- Генериране: replicate *n c*
- Стрингове: words (string -> list), unwords(list -> string), lines (string with \n as del -> list)
- Функции: flip (обръща реда на арг. във функцията), maximumBy, groupBy

mid (a, b, c) = max a (min b c) applyAll fs x = foldr (\f acc -> f acc) x fs	isLeapYear y = (y `mod` 4 == 0 && y `mod` 100 /= 0) || y `mod` 400 == 0	distance (x1, y1) (x2, y2) = sqrt ((x2 - x1)^2 + (y2 - y1)^2)

merge xs [] = xs
merge [] ys = ys
merge (x:xs) (y:ys)
  | x <= y = x : merge xs (y:ys)
  | otherwise = y : merge (x:xs) ys
	countDigits n
	  | n < 0 = countDigits (abs n)
	  | n < 10 = 1
	  | otherwise = 1 + countDigits (n `div` 10)
		compress [] = []
		compress [x] = [x]
		compress (x:y:xs)
		  | x == y = compress (y:xs)
		  | otherwise = x : compress (y:xs)

isPrime n
  | n <= 1 = False
  | otherwise = checkDiv 2
  where
  checkDiv d
  | d * d > n = True
  | n `mod` d == 0 = False
  | otherwise = checkDiv (d + 1)
	isBalanced str = helper str 0
	
	helper [] counter = counter == 0
	helper (x:xs) counter
	  | counter < 0 = False
	  | x == '(' = helper xs (counter + 1)
	  | x == ')' = helper xs (counter - 1)
	  | otherwise = helper xs counter
		encode [] = []
		encode (x:xs) =
		(1 + length (takeWhile (== x) xs), x)
		: encode (dropWhile (== x) xs)
		
		decode [] = []
		decode ((n, x):xs) =
		replicate n x ++ decode xs

removeAt _ [] = [] removeAt 0 (_:xs) = xs removeAt index (x:xs) = x : removeAt (index - 1) xs	rotateLeft rotations l = drop rotations l ++ take rotations l rotateRight rotations l = drop (length l - rotations) l ++ take (length l - rotations) l	maximumBy' _ [] = error "empty list" maximumBy' cmp (x:xs) = foldr (\y acc -> if cmp y acc == GT then y else acc) x xs

partition' _ [] = ([], [])
partition' pred (x:xs)
  | pred x = (x:l, r)
  | otherwise = (l, x:r)
  where
  (l, r) = partition' pred xs
	groupBy' _ [] = []
	groupBy' f (x:xs) = (x : ys) : groupBy' f zs
	  where
	  ys = takeWhile (f x) xs
	  zs = dropWhile (f x) xs
	ex. groupBy' (==) [1,1,2,2,2,3,3,1]
		hasDuplicates [] = False
		hasDuplicates (x:xs) = x `elem` xs || hasDuplicates xs
		
		inCircle' (a,b) (c,d) r = (a-c)^2 + (b - d)^2 < r^2

subsequences' [] = [[]]
subsequences' (x:xs) =
  sortOn length (map (x:) rest ++ rest)
  where
  rest = subsequences' xs
	quickSort [] = []
	quickSort (x:xs) =
	  quickSort [y | y <- xs, y <= x] ++
	  [x] ++
	  quickSort [y | y <- xs, y > x]
		histogram s = [(c, count c) | c <- nub s]
		  where
		  count c = length [x | x <- s, x == c]
		
		intersect' xs ys = [x | x <- xs, x `elem` ys]

mostFrequent l = selectMax (histogram l)
  where
  selectMax [(c,_)] = c
  selectMax ((c,n):(d,m):rest)
| n > m = selectMax ((c,n):rest)
| n < m = selectMax ((d,m):rest
| otherwise = selectMax ((min c d, n):rest)
 	
findMin [(_, q)] = q
findMin ((_,q):xs) = min q (findMin xs)
	intToList x
	  | x < 10 = [fromIntegral x]
	  | otherwise = intToList (x `div` 10) ++ [fromIntegral (x `mod` 10)]
	fromDigits :: [Int] -> Int
	fromDigits [] = 0
	fromDigits L = foldl (\acc x -> acc * 10 + x) 0 L
	myNub [] = []
	myNub (x:xs) = x:myNub(filter (/= x) xs)
		index el list = helper el list 1
		  where
		  helper _ [] _ = -1
		  helper el (x:xs) i
		  | el == x = i
		  | otherwise = helper el xs (i + 1)
		
		groupsof [] _ = []
		groupsof xs n = take n xs : groupsof (drop n xs) n